VIET NAM NATIONAL UNIVERSITY HO CHI MINH

UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY

Final Project

Super Mario Game

Course: Object-Oriented Programming - CS202

Student list:

Nguyễn Đình Minh Huy (25125083)

Trần Gia Huy (25125084)

Group 52 - Class 25A01

31st August 2026

Contents

1 Abstract

This report documents a 2D side-scrolling platformer in the style of *Super Mario Bros.*, written in C++17 with SFML 3.0.2 and ImGui-SFML. The project's purpose is not the game as such but the architecture behind it: a four-level abstract inheritance tree over 13 enemy, 13 item, 10 block and 5 player classes, driven polymorphically by a fixed-timestep engine, and organised by ten software design patterns. The deliverable is 27,926 lines across 130 translation units and 139 headers, with 23 verification harnesses, 13 of which are registered with CTest and run by CI on every push.

Two things distinguish this report from a feature list. First, every capability claimed here has been checked to be **reachable from `main()` and observed running** — a class that compiles and a harness that constructs it are not evidence, and the project has a documented history of confusing the two. Second, §12 states plainly what is *not* done, including one rubric line the project deliberately forfeits.

2 Introduction and objectives

The assignment asks for a 2D Mario-style game demonstrating inheritance, encapsulation, polymorphism and abstraction, with at least five design patterns, three levels of increasing difficulty, collision detection, sound, state management and save/load. We proposed and were granted an expanded scope of 110 features, on the argument that a larger surface is what makes the patterns *load-bearing* rather than decorative: a Factory that builds three types is a switch statement with a nice name; one that builds 41 types, configured from JSON, is the reason the level loader does not know what a Goomba is.

Three engineering objectives shaped the result:

1. **No entity may know about the world.** An enemy that wants to throw a hammer cannot reach the entity list; it publishes an event. This one constraint is what keeps the dependency graph acyclic, and §10.1 describes what happened when the mechanism behind it was used carelessly.

2. **Behaviour is composed, not inherited.** Movement is a swappable strategy, player form is a state object, temporary power-ups are decorators around it. Adding "a Koopa that flies" is a constructor argument, not a subclass.

3. **Nothing is complete until it has been seen running.** Enforced by CI, and by a rule that a checkbox may not be ticked on the basis that a file compiles.

3 Group information and contributions

Work was split into two *vertical* domain slices rather than horizontal layers, so that both members wrote engine code and presentation code rather than one owning systems and the other owning UI.

Member	Domain	Systems work	Presentation work	Commits
A — Nguyễn Đình Minh Huy 25125083	Player & World	Core engine and game loop, physics and collision pipeline, player entities and states, items, TileMap and level loading, camera, save/load, time rewind, map generator	Parallax backdrop, screen transitions, Menu / WorldMap / Playing / Options states, two-player modes, level editor	179
B — Trần Gia Huy 25125084	Enemies & Interaction	Input manager and command objects, sound manager, enemies and AI movement strategies, blocks, entity factory, object pool, replay recorder, debug console	Sprite sheets and animation, HUD, minimap, particles, death effects, audio wiring, character select, pause / game over / victory, statistics, achievements, boss fights	57

Commit counts are from `git shortlog -sne` on the integration branch and are a measure of activity, not of value: a large share of Member A's total is engine refactoring and defect work, which produces many small commits. Both members' work is present in every subsystem the game runs.

4 Requirements coverage against the rubric

Each row names where the capability lives and how it was confirmed. "Observed" means the game was run and the behaviour seen; "test" means a CTest case exercises it.

Rubric item	**Status**	**Where it lives, and the evidence**
Player inputs, movement, collision *(20)*	Done	`InputManager` + 8 command objects; `PhysicsEngine` with a fixed 1/60 s step, coyote time, jump buffering, wall slide, ground pound; `SpatialHash` broad phase into AABB narrow phase and `CollisionResolver`. Observed; covered by `verify_regressions` and `verify_all`.
Enemy behaviour *(10)*	Done	13 enemy classes including two bosses, driven by 8 interchangeable `IMovementStrategy` implementations (patrol, chase, fly, tethered chase, hammer throw, timer emergence, linear, proximity trigger). Observed; `verify_enemies`, `verify_enemies_new`.
Power-ups and items *(10)*	Done	13 item classes; five player forms as State objects plus two Decorators (Star, Mega). Observed; `verify_blocks`, `verify_regressions`.
Three levels *(15)*	Done	1-1 Grassland, 1-2 Ice Cavern, 1-3 Bowser's Castle, plus a bonus stage and three pipe-reachable sub-levels. All are JSON, loaded through `LevelLoader`; 1-3 ends in a boss fight. Observed end to end.
Sound *(10)*	Done	29 effects and 12 music tracks. `SoundManager` subscribes to 19 event types, so audio is wired by the Observer bus rather than called from gameplay code. Observed; `verify_sound_visual`.
Object-oriented design *(10)*	Done	§6. Four-level abstract hierarchy, private state with action-oriented methods rather than trivial accessors, and polymorphic dispatch through `Entity*` in every engine pass.
Five design patterns *(25)*	Ten	§7 names all ten with their file and the problem each solves.

Rubric item	**Status**	**Where it lives, and the evidence**
AI *(5, additional)*	Done	Two tiers. Per-enemy AI through movement strategies, and a full CPU *opponent*: `AIController` builds an observation of the world each frame and drives a second Player through the same command objects a human uses, with selectable skill and archetype. Observed — §11, figure 4.
Multiple players *(5, additional)*	Done	Four modes: Versus (two humans), Versus CPU, Co-op (shared lives and score, friendly fire off) and Shadow Chase (one human pursued by a 3-second-delayed replay of themselves). Independent key bindings per player; shared-screen camera framing the midpoint. Observed.
3D game *(5, additional)*	Not attempted	**Deliberately forfeited.** The brief offers 2D or 3D, and the project chose 2D: rebuilding the renderer in 3D would have consumed the effort that went into the pattern architecture without demonstrating any more OOP. We record this as a real 5-point cost rather than describing the parallax backdrop as "2.5D".
Save / load *(requirement)*	Done	`Serializer`, JSON, three slots plus auto-save at checkpoints; separate files for settings, high scores and campaign progress. `verify_save_load`.
Game state management *(requirement)*	Done	Eight `IGameState` implementations on a stack-capable `GameStateManager`. `verify_frontend_states`.
Level editor *(bonus)*	Done	In-game ImGui editor (F1): full tile and entity palette, free camera, undo/redo via Command objects, JSON export/import. Observed — figure 5.
Character switching *(bonus)*	Done	Four playable characters with distinct physics (Luigi jumps 1.2× higher, moves 0.85×, falls 0.9×, and double-jumps) and a selection screen.

5 Project architecture

The engine is layered, and the dependency arrows point one way only. Nothing in `Entities` includes anything from `Core` except the singletons and the event bus; nothing in `Physics` knows what a Goomba is.

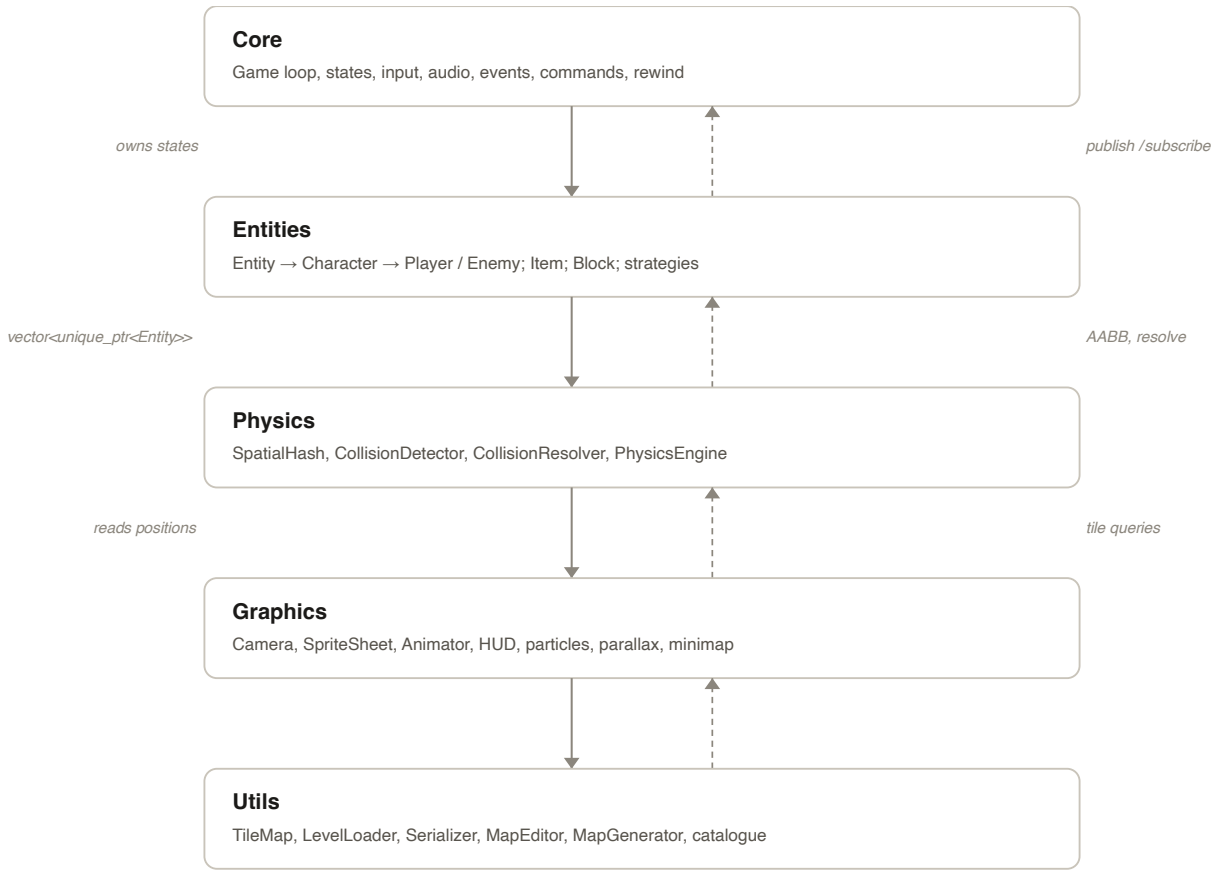

Figure 5 — The five-layer architecture.
Solid arrows are the ownership/data direction (Core owns the states that hold entities; entities are read as a flat list by physics; physics writes positions graphics then draws). Dashed arrows are the narrower channel each layer answers back through — an event, a resolved collision, a tile lookup — never a direct call in the other direction.

The frame is a fixed-timestep loop: input is drained from the OS event queue, the accumulator runs `update(1/60)` as many times as it owes, and rendering interpolates between the last two states. Physics is therefore deterministic and independent of frame rate, which is what makes the Memento-based time rewind (§7) reproducible.

6 OOP design

Every diagram in this section is **generated from the headers** by `SuperMarioGame/tools/gen_class_diagram.py` at the moment this report is built, so it cannot drift from the code. That is not a stylistic preference: the project's previous hand-written `class_diagram.md` had rotted to the point of omitting `Boss`, `Bowser`, `BoomBoom`, `Spiny`, `Lakitu`, `ShadowMario`, `AIController`, `ObjectPool` and `TimeRewindManager` — most of the architecture worth drawing — while still reading as authoritative. It is now generated by the same script.

A dashed border and a ≪*stereotype*≫ tag mark an abstract class or an interface; the hollow triangle is UML generalization, pointing at the base.

6.1 The entity tree

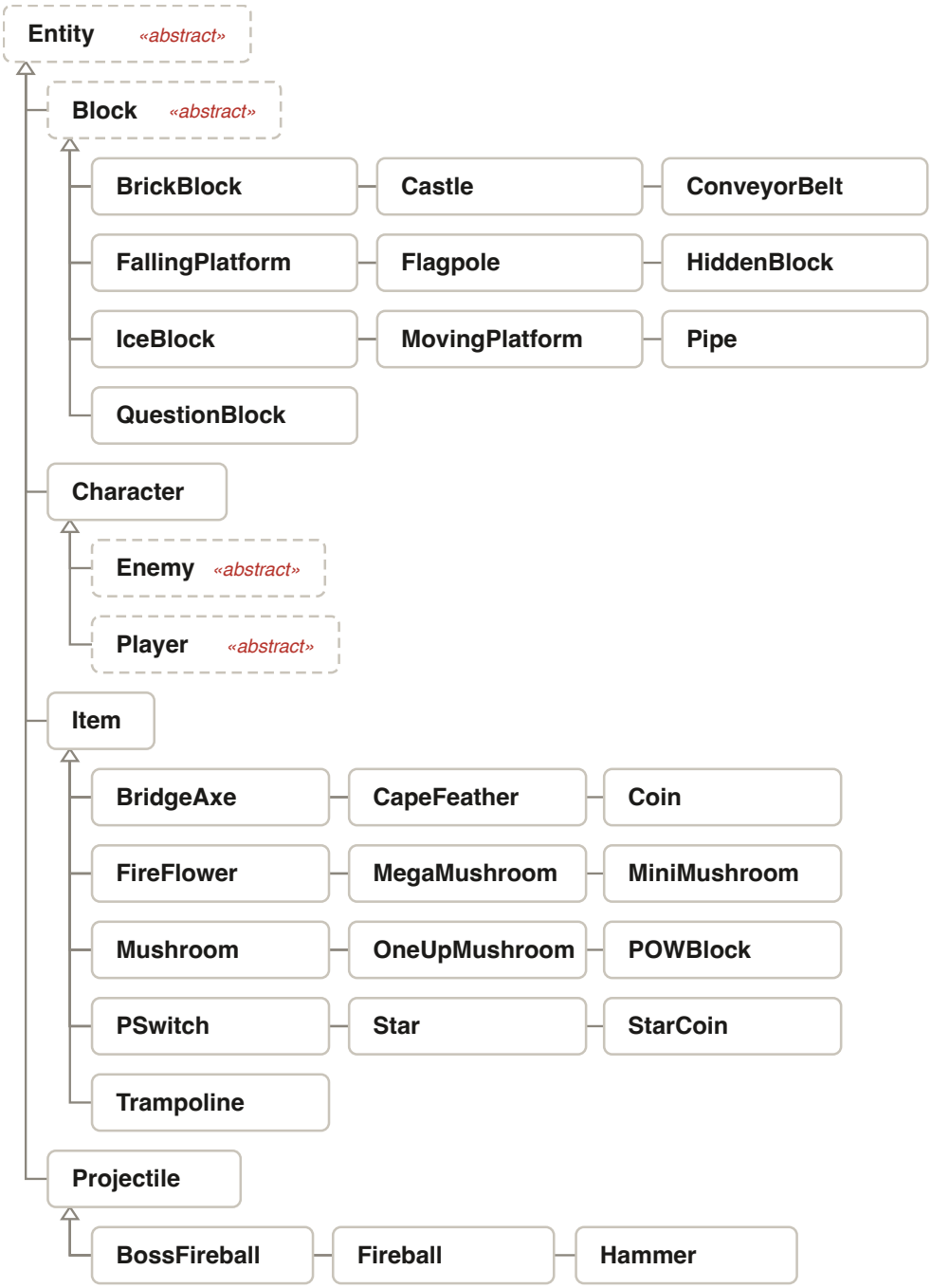

Figure 6 — The Entity hierarchy, two levels deep.

Four branches under one abstract root. Character adds motion state; Item adds activate/collect; Block adds onHitFromBelow; Projectile carries no gravity. The engine holds all of them as Entity.*

The root is abstract in the strict sense — `Entity::update`, `Entity::render` and `Entity::getBoundingBox` are pure virtual, so no leaf can exist without deciding what it does. The three second-level abstracts

each add exactly one obligation: `Item::activate(Player&)`, `Block::onHitFromBelow(Player&)`, and `Character`'s velocity and grounded state.

6.2 The other two branches: Item and Block

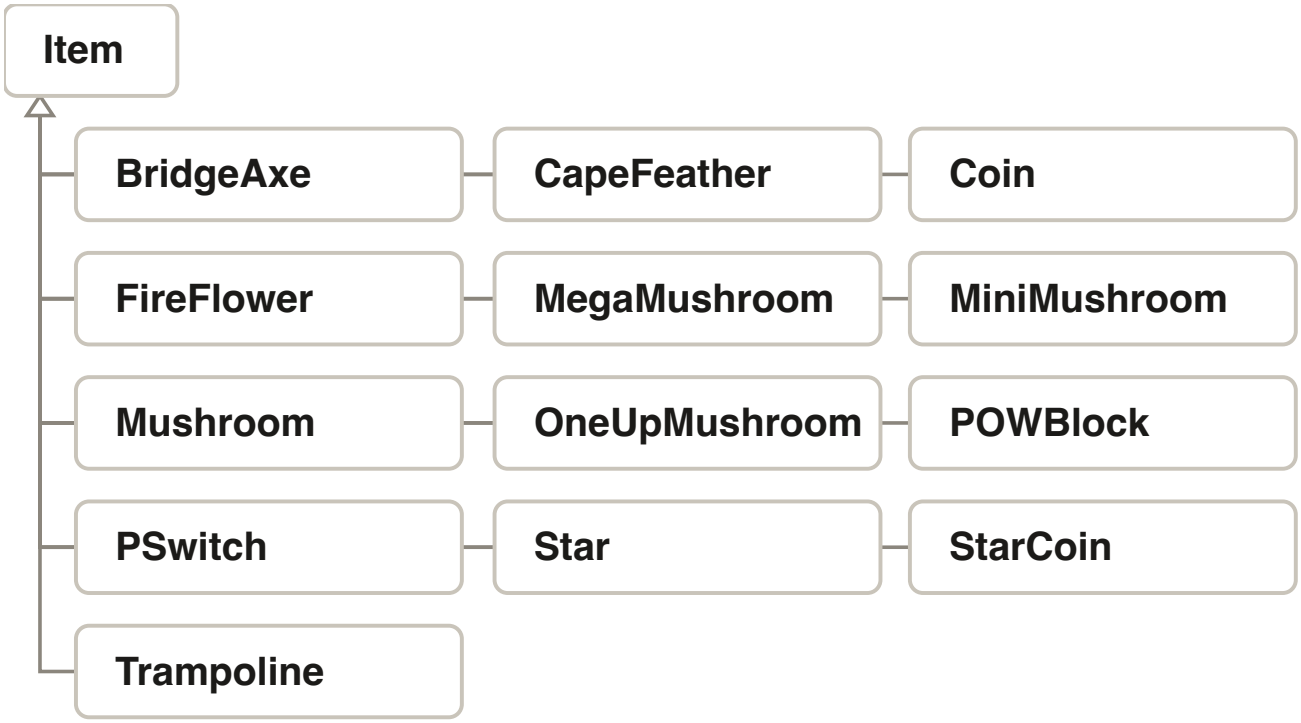

Figure 6a — Twelve items behind one activate() call.

A Mushroom, a Star and a 1-Up all answer the same question — what happens to the player who touches me — with a different body. QuestionBlock never asks which one it is holding.

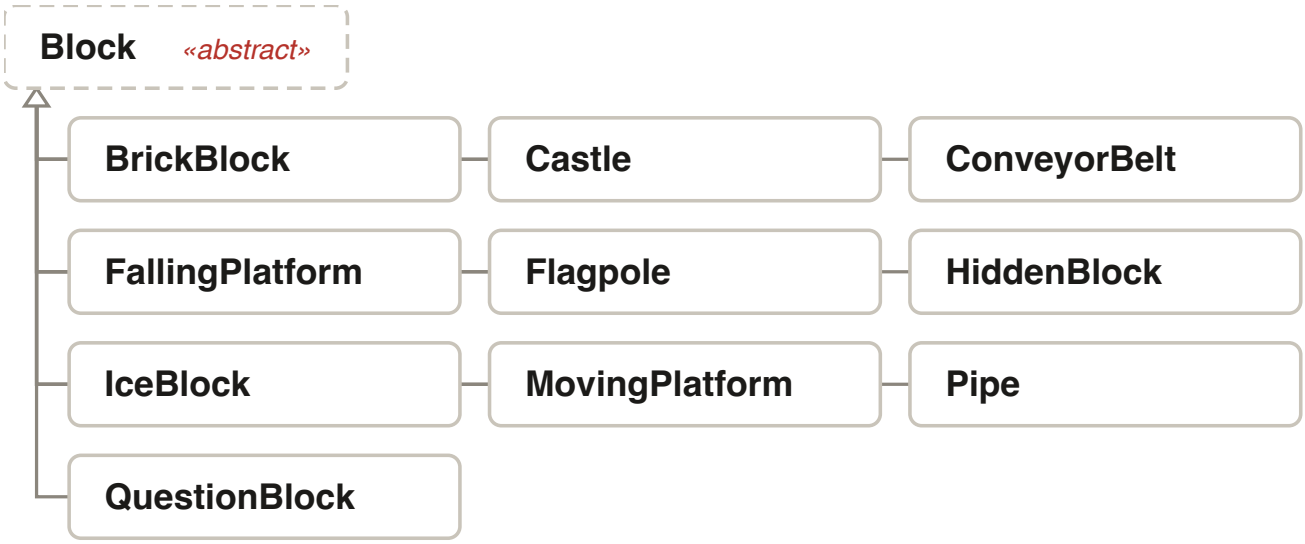

Figure 6b — Ten blocks, three of them stateful.
FallingPlatform and Thwomp are State machines in their own right (Idle/Shaking/Falling/Respawning; wind-up/slam/rest/climb); Pipe and Flagpole are not — they fire once. Block does not know the difference, which is the point of putting it here rather than in each concrete class.

`Item` and `Block` look similar — both are collidable, both are mostly inert until the player reaches them — and the project's own history has a concrete argument for keeping them as separate branches rather than merging them into one "InteractableEntity". A block's defining question is *what is on the other side of this collision, structurally* (`collidesWithTiles()`, `onHitFromBelow`) — the physics engine needs to know before it resolves a frame. An item's defining question is *what happens to the player who reaches it* (`activate(Player&)`) — physics does not need to know in advance, because touching a Star does not change how the world resolves collisions this frame. Collapsing the two would have handed the physics engine's collision-resolution code a branch on "is this actually solid" for every entity that is nominally a block, which is exactly the kind of type-testing OOP's dispatch is supposed to replace. `PSwitch` is the edge case that proves the split is real work and not just naming: it is a `Block` (grounded, collidable) whose `onHitFromBelow` triggers a level-wide event rather than reacting locally — still a block, because the physics engine still needs to resolve a jump-bump against it, but the one that most resembles an item.

Ten concrete blocks and twelve concrete items ship in this project, twice what the rubric's five-item minimum would need per SPEC's rubric-alignment reasoning (§2) — the same doubled-scope argument the feature list makes, made visible in a diagram instead of a count.

6.3 Characters, enemies and bosses

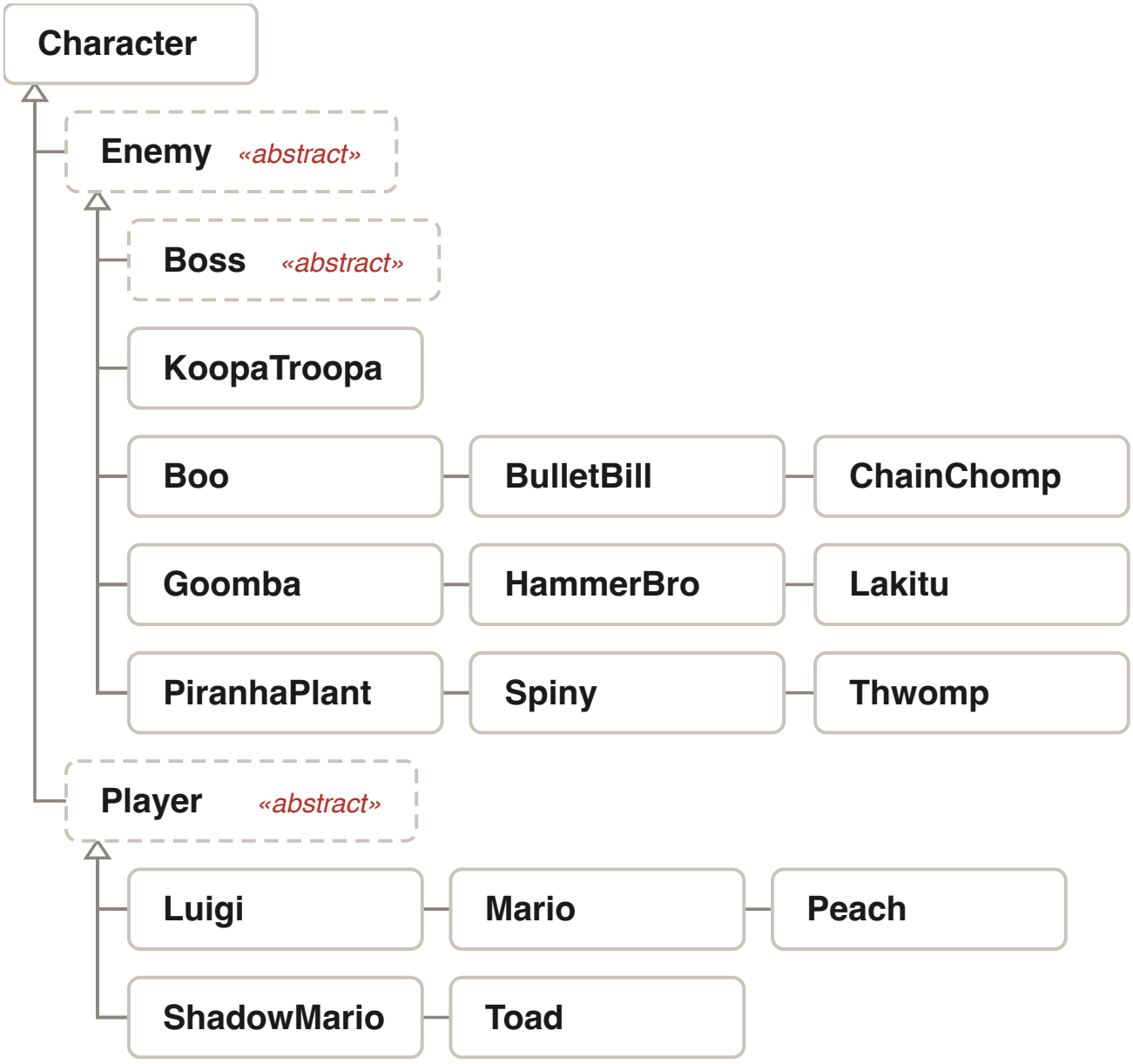

Figure 7 — Character splits into Player and Enemy.

Five playable characters including ShadowMario, the delayed replay of the human player used by Shadow Chase mode. Enemy is where the movement strategy is held.

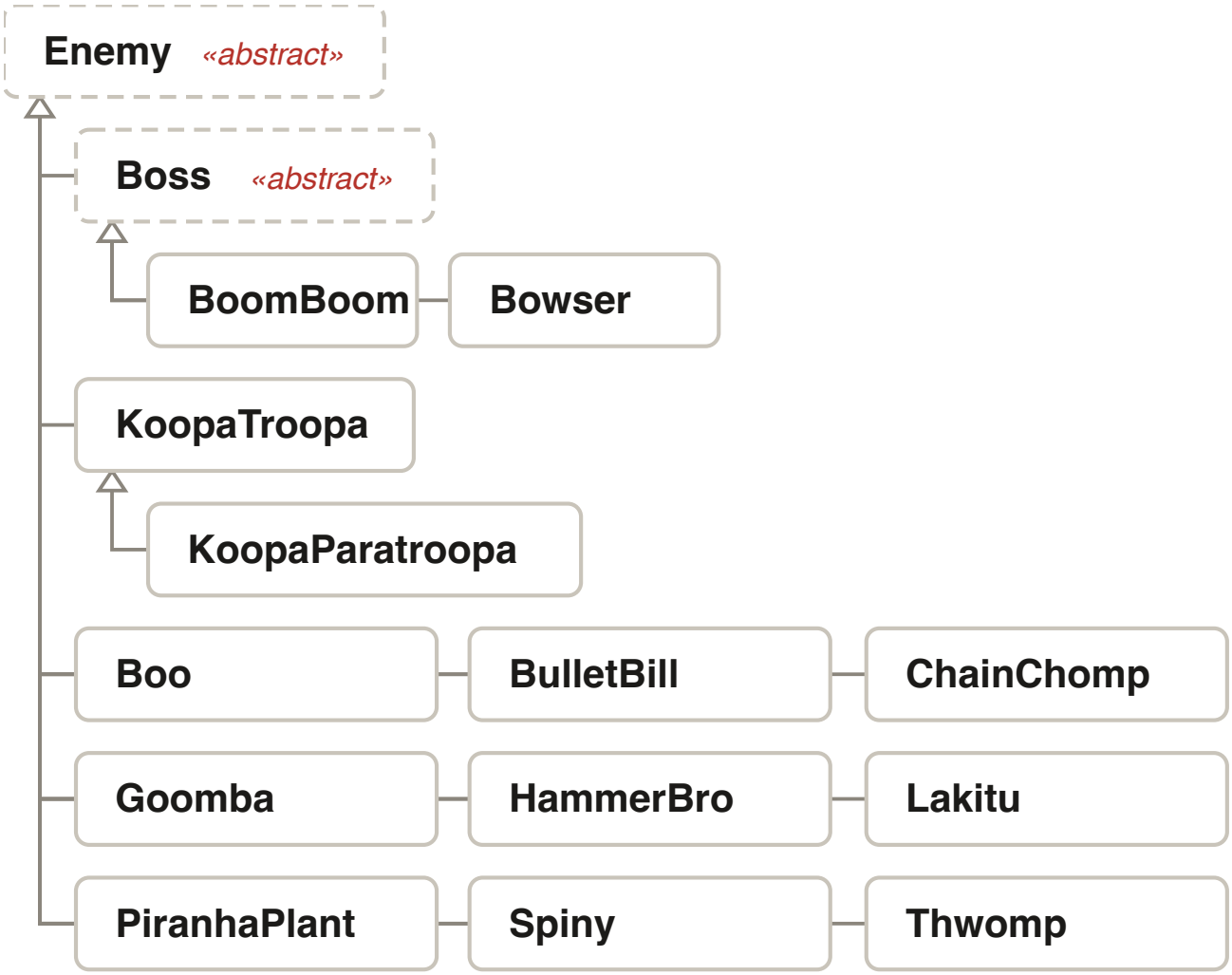

Figure 8 — The enemy branch, with the Boss sub-hierarchy.
Boss inserts a whole layer between Enemy and its two concrete fights: health bar, phases, i-frames, arena and stagger window, sequenced by a final update() so no boss can skip them.

`Boss` is the clearest argument in the codebase for a deep hierarchy rather than a flag. It is not "an enemy with more health": it seals `update()` as `final` and calls a pure-virtual `updateBehaviour()`, so *every* boss gets its invulnerability frames, phase transitions and defeat sequence in the correct order and a new one writes only its attack pattern. Bowser adds fire breath, a phase-2 leap and the fireball-stagger window; BoomBoom adds a charge. Neither can forget the rest, because neither is given the chance.

6.4 Player form: State plus Decorator

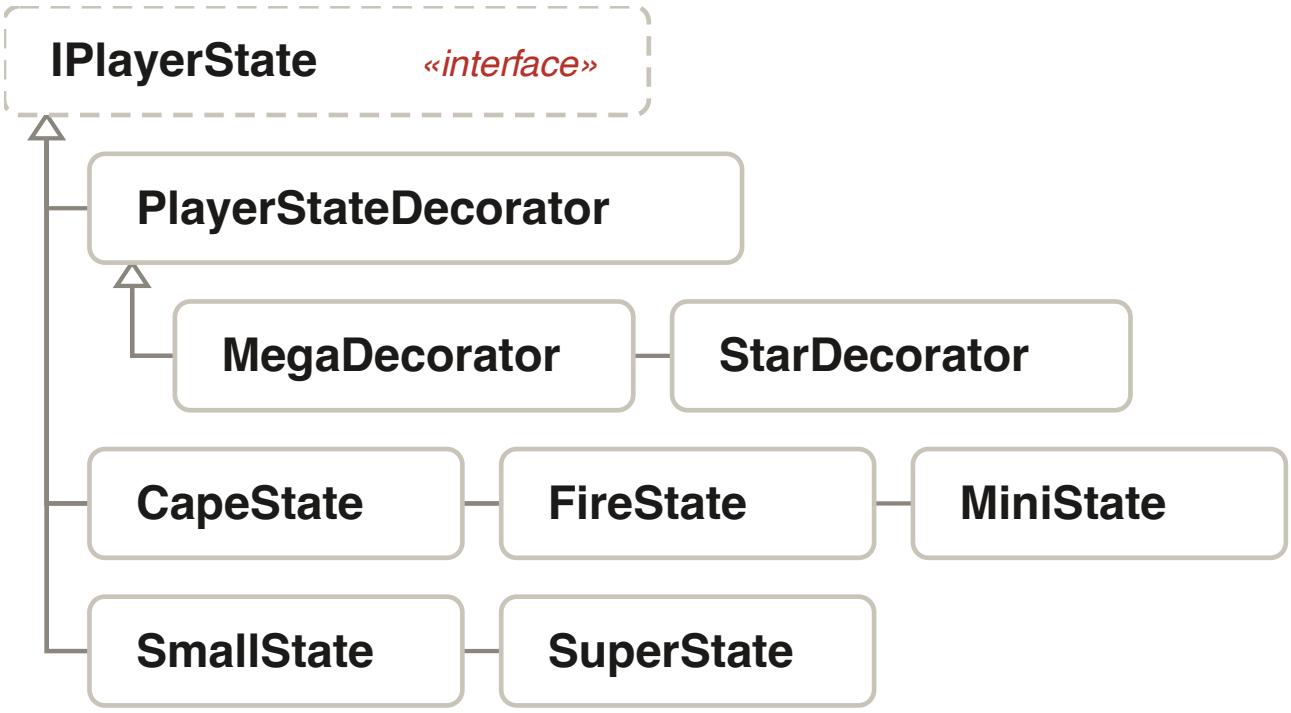

Figure 9 — Five forms and two decorators.
PlayerStateDecorator wraps whatever base state is active, which is what lets a Star be temporary and orthogonal: Fire Mario with a Star is still Fire Mario underneath.

Form changes Mario's jump height, hitbox and reaction to damage. As five state objects plus a transition table, that lives in one place; as booleans it was a growing pile of conditionals. The decorators are the part worth noticing — a sixth "Star state" would have to remember which form to exit back into, whereas wrapping preserves it for free.

6.5 The pattern interfaces

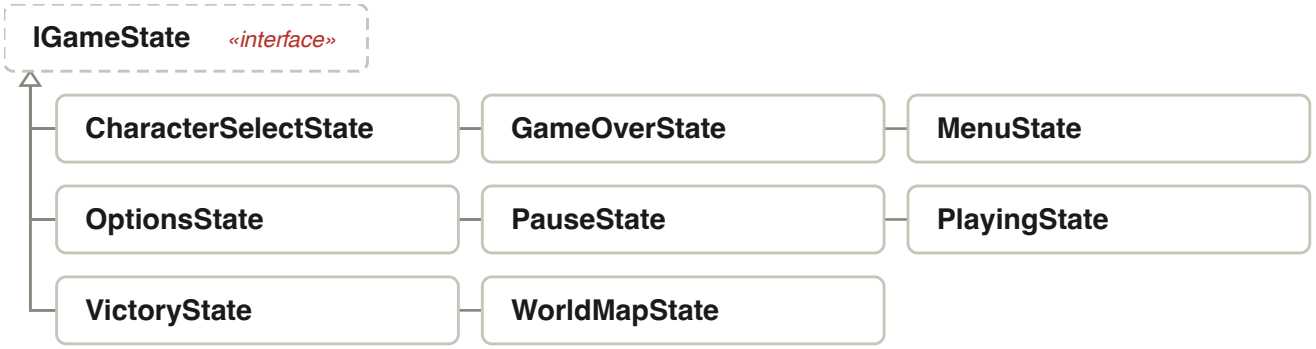

Figure 10 — Eight screens behind one interface (State).

GameStateManager holds a stack, so PauseState can overlay PlayingState rather than replace it.

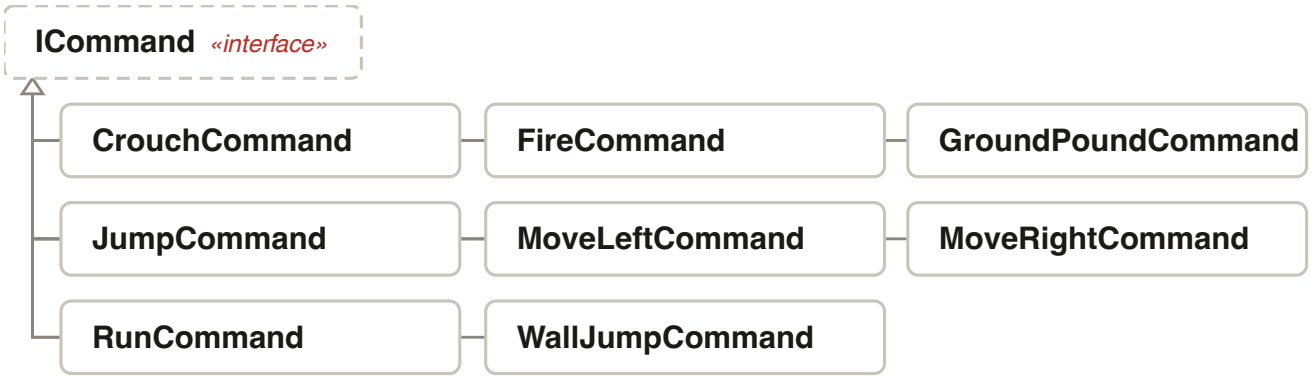

Figure 11 — Input as objects (Command).

Every action the player can take is an object, which is what makes key rebinding possible and lets a second player use the same commands through a different binding table.

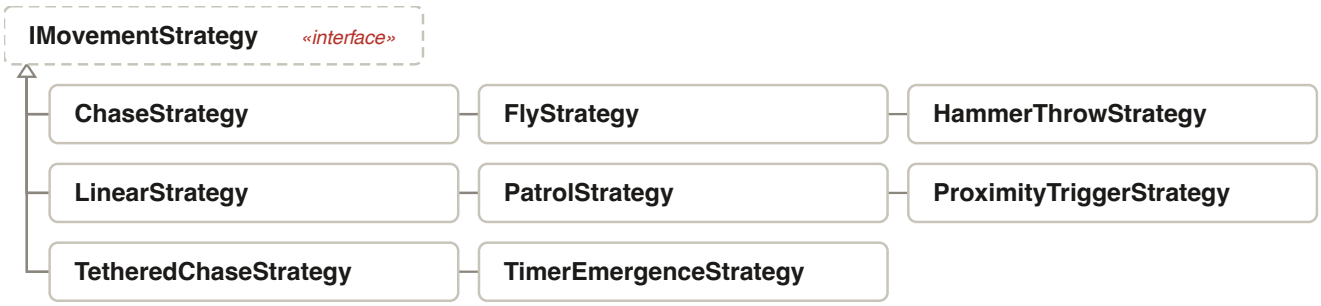

Figure 12 — Eight interchangeable movements (Strategy).
A Koopa and a Paratroopa differ only in which of these they hold, so "the same enemy but flying" is a constructor argument rather than a subclass.

6.6 Polymorphism where it earns its keep

The engine never asks what an entity is. `PhysicsEngine::update` takes `vector<unique_ptr<Entity»` and calls virtual methods: `getGravityMultiplier()` returns 0 for a block and 1 for a Goomba, so one integrator handles both; `collidesWithTiles()` returns false for a dying player, which is what lets a corpse fall through the floor without a single special case in the physics code.

The one place a `dynamic_cast` chain used to exist — a 30-branch function turning an entity into its serialised name — was replaced by a virtual `getTypeName()`. That change also fixed a live defect: the chain tested base classes before derived ones, so several types were shadowed by their parent and saved to disk under the wrong name.

6.7 Encapsulation

State is private or protected and reached through action-oriented methods: `player.takeDamage(1)` rather than a settable health field; `boss.tryStomp()`, which returns whether the hit actually landed, rather than a public health counter the resolver would have to interpret. Where a getter exists it answers a question the caller has a right to ask (`isInvulnerable()`, `isStaggered()`), never handing out an internal container or a raw window pointer.

Two deliberate exceptions, both narrow. The physics classes are declared `friend` of `Entity` so they can write positions during resolution — preferred over public position setters that any code could reach. And `Serializer`'s file-path helpers stay private even though the tests wanted them: the tests were given `Serializer::clearHighScores()` instead, because what they needed was the table cleared, not the path it lives at.

7 Design patterns

Ten, each solving a problem the codebase actually had. The rubric asks for five.

Pattern	Where	The problem it solves
Factory	`EntityFactory::create`	Levels are JSON. The loader reads the string `"koopa_paratroopa"` and must produce an object without including its header. The factory is the single construction point for all 41 types, and applies `entities.json` tuning on top.
Singleton	13 managers: `Game`, `ResourceManager`, `SoundManager`, `InputManager`, `EventBus`, `AchievementManager`, ...	One audio device, one texture cache, one event bus. Meyers singletons, so construction order is defined and no global is initialised before `main()`.
Observer	`EventBus`, 35+ event types	Collecting a coin must update the HUD, play a sound, advance statistics and possibly unlock an achievement. Without the bus, `Coin::activate` would have to know about all four. Subscriptions are tombstoned rather than erased so a handler may unsubscribe during delivery.
State	`IGameState` (8 screens); `IPlayerState` (5 forms); `FallingPlatform`, `Thwomp`	Mario's jump height, hitbox and reaction to damage all change with his form. As states, the transition table lives in one place; as flags, it was a growing pile of conditionals.
Strategy	8 `IMovementStrategy` implementations	A Koopa and a Paratroopa differ only in how they move. Composition means "the same enemy but flying" is a constructor argument.
Command	8 input commands; `IEditorCommand`; `IConsoleCommand`	Rebindable keys, and an editor whose undo/redo is free because every edit is already an object that knows how to reverse itself.

Pattern	Where	The problem it solves
Decorator	`StarDecorator`, `MegaDecorator` around `IPlayerState`	A Star is temporary and orthogonal to form: Fire Mario with a Star is still Fire Mario underneath. Wrapping the active state preserves what it wraps; a sixth state would have to be exited back into the right one.
Memento	`GameSnapshot`, `TimeRewindManager`, `ReplayRecorder`	Hold R to rewind five seconds. Entities are restored *by id*, not by index, because pruning and spawning permute the list between capture and restore.
Object Pool	`ObjectPool<T>` for fireballs, hammers, boss fireballs	Projectiles are created and destroyed several times a second. The pool trades in `unique_ptr`, so pooled and unpooled entities are stored identically and the entity list never learns that pooling exists.
Template Method	`Boss::update` is `final` and calls `updateBehaviour()`; `IMovementStrategy::execute`	Every boss needs i-frames, phase transitions and a defeat sequence in the right order. Sealing the skeleton means a new boss writes only its attack pattern and *cannot* forget the rest.

7.1 The naive alternative, for four of them

"The rubric asks for five" is not a reason to use a pattern; the table above names the actual problem each one solved. Four are worth walking through the alternative that was rejected, because the alternative is not a straw man — it is what an early, smaller version of this codebase actually looked like before the pattern replaced it.

Factory. The naive alternative is a level loader that `#include`s every concrete entity header and switches on the type string: `if (name == "goomba") return new Goomba(pos); else if (name == "koopa_troopa") ...`. It works for the first ten types. By 41 types it is a 41-branch function that every new entity must be added to, in a file (the level loader) that has no other reason to know a Spiny exists. `EntityFactory` moves that one decision into one file whose entire job is making that decision, and the loader asks a question ("build me a Spiny") instead of making one.

Observer. The naive alternative is `Coin::activate(Player& p)` calling `p.addCoins(1); hud.`

`refresh(); soundManager.play("coin"); stats.recordCoin(); achievements.checkCoinMilestones(` directly — and then `Star::activate`, `OneUpMushroom::activate` and every other item repeating some subset of the same list, because each item's designer has to remember which systems care this time. `EventBus::publish` lets `Coin::activate` know only that something happened, not who is listening — and a fifth system (say, a future daily-challenge coin counter) subscribes without `Coin.cpp` changing at all.

State + Decorator, together. The naive alternative is not a competing pattern but the thing both of these replaced: a pile of booleans on `Player` — `isSuper, isFire, isCape, isMini, isStarred, isMega` — and an `if/else` chain in every place form matters (hitbox, jump height, death behaviour, rendering). The moment a Star is picked up as Fire Mario, that boolean pile has to encode "currently Fire, but also temporarily starred" as a combination no single flag names, and every one of those `if` chains has to be taught the combination separately. Five state objects handle the base forms; two decorators wrap whichever one is active without either of them needing to know what they are wrapping. The combination was never a special case to design for — it falls out of the two patterns being orthogonal to begin with.

Template Method. The naive alternative is copying `Bowser`'s `update()` into `BoomBoom` and editing the attack pattern in place — which is exactly how the second boss *would* have been built without this pattern, and is the standard way an i-frame check or a phase-transition line quietly diverges between two copies over several edits. Sealing `Boss::update()` as `final` is a compiler-enforced version of "do not copy this function": a third boss can only be added by writing `updateBehaviour()`, which is not capable of skipping the sequencing the base class already owns.

8 Implementation details

8.1 The frame

A fixed-timestep accumulator: input is drained from the OS queue, `update(1/60)` runs as many times as the accumulator owes, and rendering interpolates between the last two states. Physics is therefore deterministic and independent of frame rate — which is what makes the Memento rewind (§8.7) reproducible, and what lets the CPU opponent be judged against the same simulation a human plays.

```
Game::run()
poll OS events ............ InputManager records held keys from the event STREAM,
not from sf::Keyboard::isKeyPressed - see 8.3
accumulator += frameTime
while accumulator >= 1/60:
PlayingState::update(1/60)
0.  rewind check (R held) -> restore a Memento and return
1.  held keys -> command objects -> Player
2.  AIController decides, for a CPU opponent
2c. Shadow Mario samples this frame's inputs
3.  for each entity: update()            <-- may REQUEST spawns
4.  PhysicsEngine::update(entities, tilemap)     see 8.2
5.  flushPendingSpawns()                 <-- only point the list may grow
6.  prune inactive; recycle pooled types into their ObjectPool
7.  hazards: lava underfoot, the void plane, warp pipes, boss arena
8.  P-Switch clock, HUD sync, boss HUD
9.  record one Memento
accumulator -= 1/60
render, interpolated
```

Step 5's position is not cosmetic; §10.6 is the crash that put it there.

8.2 The physics pipeline

`PhysicsEngine::update` runs ten ordered stages, and the order is load-bearing at almost every step.

Stage	What it does, and why it is here
1. Conveyor push	Applied using the *previous* frame's ground status, because this frame's has not been computed yet and a conveyor must not act on someone who has already left it.
1.1 Interactive tiles	Coin tiles are not solid, so the collision detector never reports them. The player's footprint is scanned separately — see §10.5.
1.5 Acceleration and friction	Per-character: Luigi accelerates to 0.85× the speed and keeps more momentum. Ice reduces friction; this is where surface type enters.
2. Gravity	Scaled by `getGravityMultiplier()`, so blocks (0) and projectiles (0) are skipped, water is 0.3×, and Luigi falls at 0.9×. Clamped at `TERMINAL_VELOCITY` = 600 px/s, or 60 in water.

Stage	What it does, and why it is here
3. Integrate X, resolve X	**Only the maximum horizontal overlap is resolved.** See §10.1: resolving every overlapping tile summed the push-outs.
4. Integrate Y, resolve Y	Separately from X, which is what makes walking into a wall while jumping behave correctly. Brick destruction from below happens here.
4.5 Map bounds	Everything is clamped to `[0, width × 32]`, so nothing walks off the side of the world.
5. Broad phase	`SpatialHash` with 64 px cells — two tiles, so a 32 px entity touches at most four buckets. Pairs are tested only against bucket neighbours.
6. Narrow phase and resolution	AABB overlap, then `CollisionResolver` dispatches on the pair's types: stomp, damage, collect, bump, shell kick, boss.
7. Intent flags	Cleared last, once acceleration and resolution have consumed them.

Cost. The broad phase turns an $O(n^2)$ all-pairs test into $O(n)$ expected work against bucket neighbours; tile queries are a direct index into a dense array, so $O(1)$ per probe. With a 200×23 map and a few dozen live entities the frame is dominated by rendering, not physics.

Feel. Two forgiveness windows, both six frames (100 ms): *coyote time* lets a jump register just after walking off a ledge, and the *jump buffer* lets a jump pressed just before landing fire on touchdown. Neither is physical; both are what stops the game feeling stiff.

8.3 Input: commands, and why not `isKeyPressed`

Each action is an `ICommand` object, so bindings are data: `InputManager` holds a per-player action→key table loaded from `config.json`, and Player 2 is the same commands through a second table. Rebinding is a map write.

Held state comes from the *event stream*, not `sf::Keyboard::isKeyPressed`. That is a deliberate correction: `isKeyPressed` reads global OS state, which on macOS returns false without Input Monitoring permission — so movement and the rewind key were dead for an invisible, machine-specific reason. Reading the stream also means losing window focus releases everything, rather than

leaving a key stuck down.

8.4 Entities: one door in, one door out

Everything enters the world through `EntityFactory::create()` and then `admitEntity()`, which wires its animations and applies the difficulty and New Game+ modifiers. Because that is the single door, a difficulty change reaches an entity that does not know difficulty exists.

Leaving is symmetric: the prune step uses `std::stable_partition` rather than `remove_if` — `remove_if` leaves the tail moved-from, so the dead objects would already be gone before they could be offered back to a pool. Partitioning permutes instead, and the tail is real objects. `forgetEntity()` then clears every raw pointer the state holds into the list (the active boss, the shadow, Player 2, a carried shell) *before* the `unique_ptr` behind it is released.

`ObjectPool<T>` trades in `unique_ptr<T>` rather than owning its objects, so a pooled fireball is stored exactly like an unpooled Goomba and the entity list never learns that pooling exists. The pool caps what it retains: an unbounded free list is a leak that never frees.

8.5 Enemy behaviour and the boss template

An enemy holds an `IMovementStrategy*` and delegates to it. Eight exist — patrol (turns at ledges and hazards), chase, fly, tethered chase (Chain Chomp), hammer throw, timer emergence (Piranha Plant), linear (Bullet Bill) and proximity trigger (Thwomp).

A boss is not a strategy, because a strategy describes one repeatable motion and a fight is a sequence whose behaviour depends on its own health. `Boss::update()` is `final` and sequences the shared parts — i-frame countdown, stagger clock, phase transition, defeat animation — calling the pure-virtual `updateBehaviour()` for the subclass's attack pattern. Bowser adds fire breath on a phase-dependent interval, a phase-2 leap, and the fireball-stagger window; BoomBoom adds a charge.

8.6 The tile map and level loading

A dense `vector<TileType>` of 11 types, indexed directly. Level JSON stores tile *spans* (`{"type": "ground","x":0,"y":21,"w":170}`) rather than cells, so a 200×23 level is a few dozen lines. Themes select the atlas frames and the parallax backdrop.

Loading resolves its path against several candidate roots so the game runs from either the source tree or `build/`. That fallback had a real bug worth remembering: a single `ifstream` was reused across candidates, and the first miss set `failbit`, so every later `open()` was silently ignored (§10.9). After load, three passes run over the finished level: the backdrop's ground line is derived from the widest solid row in the lower half of the map; the flagpole and castle are settled onto whatever floor is beneath them; and the void plane is set one tile below the deepest floor.

8.7 Time rewind, replay and the snapshot

`GameSnapshot` holds both players' stats, the level timer, the camera centre, and one record per entity. Holding **R** pops snapshots off a 300-frame deque — five seconds at 60 Hz — and applies them.

Two details make it correct rather than approximately correct. Entities are restored **by id**, not by index: pruning and spawning permute the vector between capture and restore, so index restoration assigned positions to the wrong entities. And applying a snapshot now *cancels an in-progress death*, because restoring a position while leaving the player in its dying state put them back above the pit still falling.

`ReplayRecorder` is the same snapshot stream kept longer and thinned, capped at 6000 frames — roughly ten minutes — because a recorder with no cap is a memory leak with a feature name.

8.8 The CPU opponent

`AIController` drives a real `Player` through the same command objects a human uses. It has no privileged access to the world, only a wider view of it: each frame it builds an `AIObservation` — a 21×15 tile grid centred on itself, each cell classified *Unknown / Empty / Solid / Hazard / Reward / Enemy*, plus normalised offsets to its objective and to the opponent, its own velocity, and three flags.

`Unknown` is a distinct state from `Empty` on purpose: difficulty is expressed as vision radius, so an Easy bot genuinely cannot see a pit it has not reached rather than seeing one and pretending otherwise. Difficulty also sets reaction latency and action noise; the archetype (Speedrunner, Hunter, Collector) sets what the objective *is*. The policy sits behind an `IAIPolicy` interface, so the heuristic implementation can be swapped without the controller changing.

8.9 Multiplayer

Four modes on one shared screen. The camera frames the *midpoint* of both players and a tether keeps them together — hard in Versus, where whoever falls behind is shoved along at the edge, and soft in Co-op, where the leader is blocked at the right edge instead so the screen simply waits. Split screen was rejected: every screen-space overlay in the game would have had to learn about viewports.

Shadow Chase is the odd one. `ShadowMario` is a `Player` replaying the human's own input stream three seconds late, with a correction lerp to stop it desyncing. It is a contact hazard and is unmoved by collision, because shoving it off its path would desync it from the recording driving it.

8.10 Graphics

Sprites come from JSON-described atlases (`player.json` alone has 92 validated frames); `Animator` plays named clips against them. The camera does look-ahead in the direction of travel, clamps to level bounds, and applies screen shake *between two clamps* so a shake cannot push the view outside the map. The parallax backdrop composes per-theme layers at fractional scroll rates, placing decorations by a hash of world position so they do not shimmer as the camera passes. Particles run on an object pool with a fixed capacity, so a fifty-coin burst allocates nothing.

8.11 The entity catalogue

One table maps every `EntityType` to its serialised name, display label and palette category. The JSON parser and the editor's palette both read it, and a CTest case asserts that every entry is constructible by the factory and that each class's own `getTypeName()` matches its catalogue name — because a mismatch means a level saved from the editor loads back as a different entity. Before this existed the list was hand-written in three places and had drifted: §10.4.

9 Data storage and serialization

Everything persistent is JSON via `nlohmann/json`, chosen over a binary format because a level file that a human can read and hand-edit is worth more during development than a few kilobytes.

File	Holds
`assets/levels/*.json`	Tile spans (run-length encoded by `w`), entity placements with per-instance fields such as Bowser's arena, spawn point, theme, dimensions.
`assets/config/entities.json`	Per-type tuning — speed, score, health — applied by the factory after construction, so balance changes need no rebuild.
`saves/slot_N.json`	Three save slots: form, lives, coins, score, position, star coins.
`saves/progress.json`	Campaign completion and unlocks; drives the world map and the New Game+ counter.
`saves/config.json`, `highscores.json`	Key bindings, volumes; the high-score table.

Tile-name and entity-name conversion each exist in exactly one function. That is a rule with a scar behind it: a duplicated string-to-enum chain in the level loader once drifted from the canonical one and silently dropped every coin tile in all seven level files.

10 Technical problems and solutions

Grouped by what they have in common rather than by the week they happened, because the groupings are the lesson. Each is drawn from the weekly reports (`docs/Group52_04/–10/`) and the session log, where the diagnosis was written down at the time.

10.1 Collision: the ground was not solid

Five separate defects, all in the first month, all producing "the player behaves oddly on flat floor". They are worth listing together because each looks like a physics-tuning problem and none of them was.

Symptom	**Actual cause**	**Fix**
The player sticks at random points while walking across flat ground	A horizontal overlap at the seam between two floor tiles was being read as a side-wall collision, so `velocity.x` was cancelled and `onWall` set.	`CollisionDetector` now looks at the tile *above* the one it hit: if that is empty, this is a floor seam, not a wall, and the push-out is converted to vertical.
`onGround` flickers true/false every frame, and the player jitters	`resolveEntityVsTile` moved `position` but not `boundingBox`. The stale box was still inside the tile next frame, re-triggering the overlap.	The bounding box is synchronised at the point of displacement.
Standing across two tiles launches the player slightly into the air	Both tile collisions resolved in the same frame and the two push-outs *summed*.	Resolve only the collision with the **maximum penetration** per axis per frame. That is sufficient: clearing the deepest overlap on a flat surface clears the shallower ones with it, so one correction does the work of all of them without double-counting.
Holding a direction against a wall in mid-air lets the player glide up it	Nothing removed vertical momentum on a mid-air wall contact.	Wall friction: upward velocity is zeroed on an airborne horizontal hit, and falling is capped at `WALL_SLIDE_SPEED` = 50 px/s — which is also where the wall-slide mechanic came from.

Symptom	Actual cause	Fix
Mario cannot enter pipes or drop through one-tile shafts	His hitbox was the full 32 px tile width, so every one-tile gap was an exact fit and caught on the corner AABBs either side.	Hitboxes narrowed and centred inside the sprite: 24×30 Small, 24×60 Super, 14×14 Mini. A 4 px margin each side is the whole fix.

The pattern: four of the five were a *bookkeeping* error — state updated in one place and not the matching one — rather than wrong physics. The fifth was a geometry assumption nobody had written down.

10.2 Gravity applied to things that should not fall

`Block` derives from `Entity`, and `Entity`'s `getGravityMultiplier()` returns 1. `Block` did not override it, so the physics engine accelerated every question block and brick downwards at 1800 px/s^2 and the level geometry fell out of the bottom of the map. The fix is one line — `return 0.0f` — but the interesting part is why it was not caught: the blocks were correct in the level file, correct on the first rendered frame, and wrong a second later.

The same defect in a different costume: a mushroom dispensed from a block had no resting state, so `resolveEntityVsTile` corrected its penetration without zeroing its accumulated downward velocity, and it eventually slipped through a seam and fell out of the world. `Item` gained an `m_onGround` flag and gravity now skips a resting item.

10.3 Object lifetime, four times

The most expensive category in the project, and the one whose members look least alike.

- **Dangling event subscriptions.** `PlayingState` subscribed to `PlayerShotFireball` and never unsubscribed, so an event published after the state was destroyed called into freed memory. Every subscription in the state now has a matching `unsubscribe()` in `exit()`.

- **An invalid-handle sentinel that was a valid index.** Subscription IDs were compared as raw integers, so an "unset" handle could address a real subscriber. The sentinel is `static_cast<size_t>(-1)`.
- **Shutdown order.** Closing the window destroyed the singleton managers while game states were still alive, and the state destructors called into them — a mutex failure on the way out. `Game::shutdown()` now pops every state *before* shutting the managers down, and closes the window before that, because SFML's OpenGL context was being torn down while the render window still held handles to it (an immediate `SIGABRT` at exit).
- **The mid-frame spawn crash** — §10.6, the culmination of this theme and the only one that was platform-dependent.

10.4 One fact, two places

The project's signature failure. In each case nothing failed to compile; the program simply did something different from what the code appeared to say.

The duplicated fact	What it cost
The tile name ↔ `TileType` mapping, re-implemented in `LevelLoader` alongside the canonical one in `SerializationUtils`	The copy drifted and silently dropped **every coin tile in all seven level files**. Both directions now live in one pair of functions, and a round-trip test guards them.
The entity type list, hand-written in the factory, the parser and the editor palette	The editor could not place a Goomba, a Koopa, a pipe or a flagpole — 16 of ~40 types, no enemies at all. Now one `EntityCatalogue` with a parity test (§8.3).
The flagpole's height: 168 px of sprite against a 300 px collision box	The pole you could touch and the pole you could see were different objects, and the catch-height score was measured against one that did not exist.
The backdrop's ground line: a hardcoded screen constant against the real tile geometry	§10.7 — it took three attempts.

The duplicated fact	What it cost
The class diagram, hand-maintained against the headers	Rotted until it omitted most of the architecture while still reading as authoritative. Now generated (§6).

The standing rule this produced: the commit that creates the second copy also creates the test that fails when the copies disagree. Writing this report exercised it — the build script counts what it claims, and caught a class count inflated by a substring match and "23 test files" being reported as "23 things CI runs".

10.5 Interaction that was never wired

Several features existed as far as the event that announced them and no further.

- **Coins on the tile grid could not be collected.** `TileType::Coin` is not solid, and the collision detector only reports solid tiles — so walking through one did nothing and the coin stayed on the grid forever. The physics engine now scans the player's footprint for non-solid *interactive* tiles after integration.
- **The coin counter ignored its own payload.** `AchievementManager` did `m_coinsThisRun++` on `CoinCollected` rather than reading the amount, so a block dispensing ten coins counted as one.
- **The P-Switch and POW block** published an event that reached a sound and nothing else — no brick/coin swap, no enemies flipped, and the HUD's countdown field was never set.
- **Six subsystems were complete and inert:** minimap, particle system, death effects and screen transitions all compiled, all had passing harnesses, and none were ever constructed by the game.

> ***The rule, not the patch.*** *A task is complete only when the code is reachable from* `main()` *and has been observed running. A harness proves a class works in isolation; it does not prove the game ever builds one. That is why §4 records how each capability was confirmed. The menu music, which had been failing silently at startup for weeks, was found by running the game for six seconds — not by any of the 86 harnesses.*

10.6 A crash on Windows that could not be reproduced on macOS

A teammate reported that the game crashed in 1-3, and that deleting Bowser from the level file made it stop. The same build played 1-3 correctly on macOS.

The cause was not in Bowser. Every spawn travels through a synchronous event: an entity that wants to create another publishes a request, and `PlayingState` performs the spawn in a subscriber. That subscriber called `m_entities.push_back()` — on the stack of the `for (auto& entity : m_entities)` loop that was calling the publisher. A range-for caches `begin()` and `end()` once, so when a `push_back` exceeded capacity and reallocated, the loop's iterators pointed into freed memory.

That is undefined behaviour, and undefined behaviour is allowed to work: the macOS allocator left the freed page mapped and readable while the Windows toolchain faulted. Bowser mattered only because he is the one entity that spawns another several times per second, so 1-3 reaches a reallocation almost immediately.

> ***Solution.*** *Spawns queue and are admitted at exactly one point per frame, after both the update loop and the physics pass have finished. Full analysis, with the amortised-growth argument and a worked trace, in* ***`docs/learning/mid-frame-entity-spawn-crash.html`****.*

10.7 A boss that could not be beaten, and a backdrop that took three tries

Bowser was near-impossible for a structural reason rather than a numeric one: he is immune to fire, a boss carries a second of invulnerability after every hit during which contact *damages the player*, and he advances breathing fire throughout. The only legal input was five clean descending stomps with no way to create an opening. Two routes were added, both from the series — four fireballs now stagger him into a three-second window where a stomp is safe and lands, and the level ends on a bridge over lava with an axe beyond it that drops both.

The backdrop is the better cautionary tale. The parallax layers drew on a hardcoded screen line of 640 px; the real ground renders at 656. The *first* fix anchored them to the lowest solid row — which is the bottom of a two-row floor slab, so they were buried a tile instead of floating one. The *second*

correctly found the top of the slab, but the renderer still decided ground-versus-sky by comparing a layer's authored baseline against the runtime ground line, so every ground layer was reclassified as sky and pinned to the stale constant anyway. Only the third fix — making "this layer stands on the ground" a property of the layer, and restricting the ground scan to the lower half of the map so a castle ceiling cannot win — was correct. It was confirmed by measuring pixel columns in captured frames, not by reading the code again.

10.8 Process failures, and the rules they produced

Three of the most costly problems were not in the code.

> ***A destructive git command deleted a week's work.*** *A session ran* `git reset --hard && git clean -fd` *to unblock a build, permanently destroying untracked files including the entire Week 8 report.* Rule: *never discard uncommitted work to unblock a git operation — committing is reversible, discarding is not — and record the before/after commit hashes of every version-control operation.*
>
> ***An audit was published from a stale checkout and had to be retracted.*** *Its first revision was written against a local* `dev` *nine commits behind, and on that basis declared the entire audio system missing. The audio system was implemented and merged.* Rule: *fetch before describing repository state; a local branch is not evidence.*
>
> ***~3,100 lines of finished work were sitting uncommitted*** *— three sub-level maps, the tools directory and a week's report — one careless clean from being lost. Found and committed during the audit.*

A fourth, found while writing this report: the test suite read and wrote the developer's real `saves/` directory, and a `ctest` run was observed **deleting actual campaign progress**. One high-score assertion also passed under `ctest` and failed running the same binary from `build/`, because the two have different working directories. Fixed at the root — every harness now redirects saves to a temporary directory, and a test asserts that containment.

10.9 Toolchain and porting

Smaller, but each cost real time.

- **SFML 3.0.2 removed `sf::Vertex`'s constructors.** Parenthesis construction no longer compiles against an aggregate; brace initialisation satisfies both 2.x and 3.x.
- **A stream that stayed failed.** `LevelLoader` tried candidate paths in a loop with one `ifstream`. The first miss set `failbit`, and every subsequent `open()` was silently ignored — so sub-levels failed to load depending only on which directory the game was launched from. One `file.clear()` per iteration.
- **Non-uniform randomness.** `rand() % range` for particle spread angles clustered visibly; replaced with `std::uniform_real_distribution` over `std::mt19937`.
- **Save files could contain nonsense.** Out-of-range player coordinates were written straight into the slot JSON and crashed on load; coordinates are clamped to the level bounds before serialisation.
- **Stale achievement toasts.** Loading a save did not clear the active toast queue, so previously-unlocked achievements popped up again after every load.

11 Features demonstration

All five figures are unretouched frames captured from the running game by a scripted input driver (`--script`), which synthesises events into the same path the OS event queue feeds. Nothing outside the process is touched, so a verification run is reproducible.

Figure 1 — The main menu, over the parallax backdrop. Seven entries including the four multiplayer modes, the map editor and a procedurally generated level.

Figure 2 — World 1-3, Bowser's Castle. The HUD reads WORLD 1-3, taken from the level catalogue

rather than a constant. The castle towers and fencing of the parallax backdrop stand on the floor the player walks on, at the correct depth tint for the castle theme.

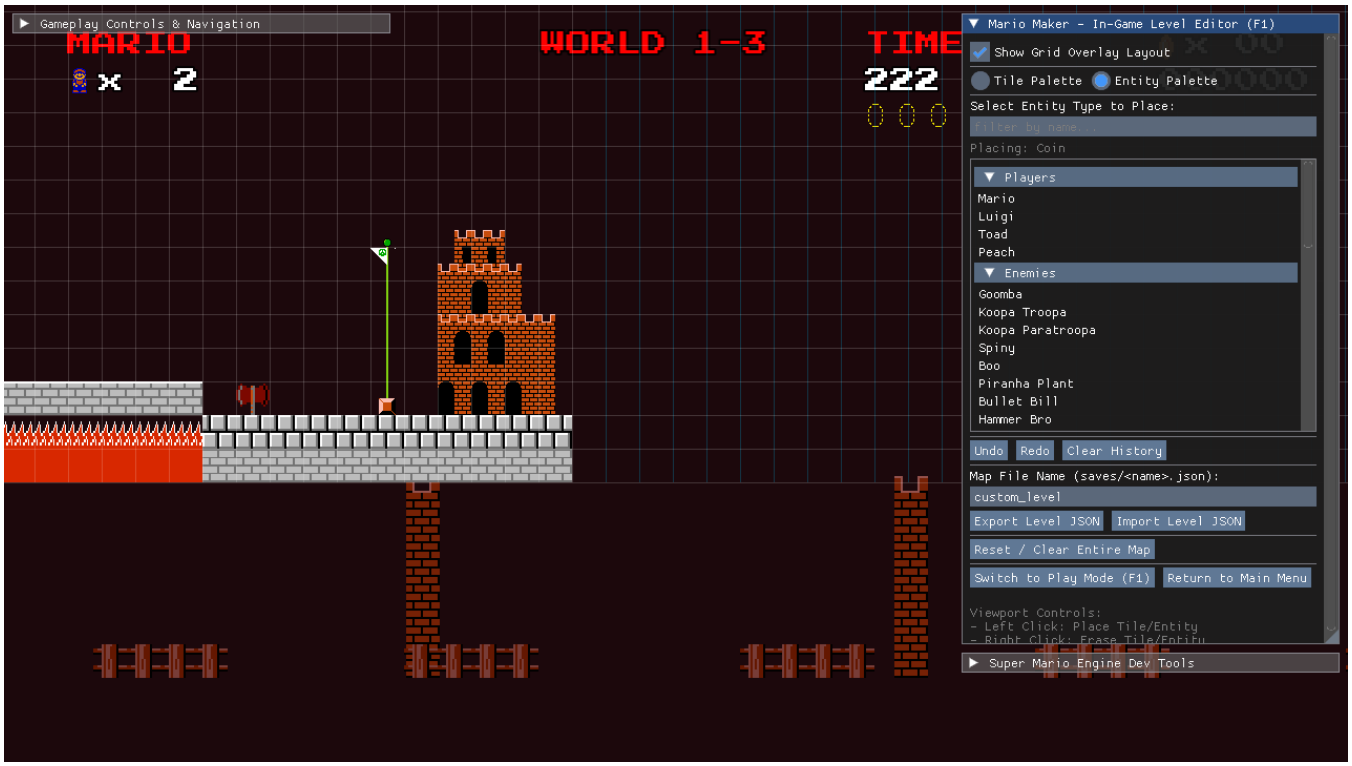

Figure 3 — The rebuilt end of 1-3, seen through the editor's free camera. Left to right: lava spanned by the brick bridge Bowser paces, the axe that drops it, the goal flagpole standing on the ground, and the castle drawn from the atlas's castle_ end art. The editor's tile palette is open on the right.

Figure 4 — Versus CPU. Both players have an icon and a life count in the HUD; the bottom strip carries the score line and who is leading. The dev panel reports the opponent's policy and what it currently believes it is doing ("crossing gap") — the CPU drives a real Player through the same commands a human uses.

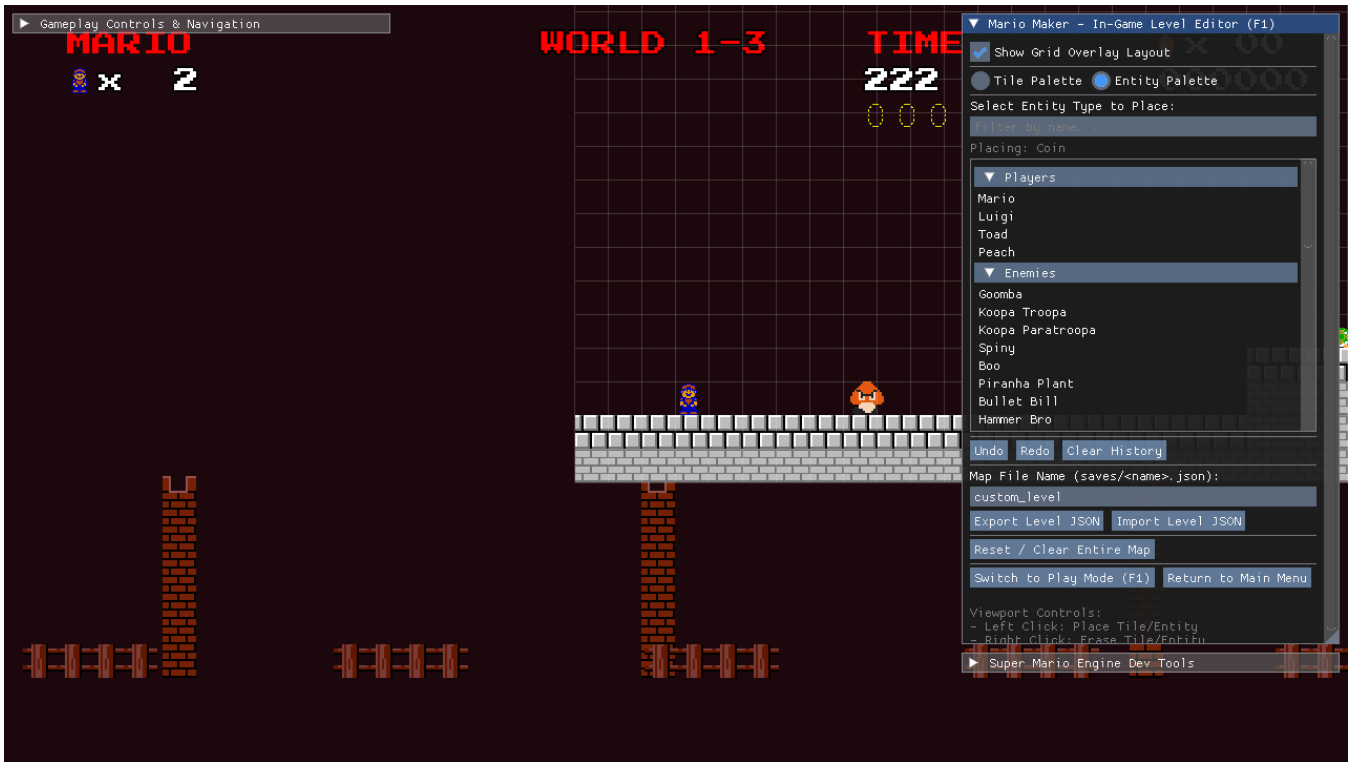

Figure 5 — The in-game level editor (F1). The entity palette is generated from the single catalogue and grouped into Players, Enemies, Items, Blocks and Scenery, with a filter box and the serialised name on hover. Edits go through Command objects, so undo and redo come for free.

12 Process and project history

The project ran from 29 May to 22 August 2026 across 7 weekly reporting periods (`docs/Group52_04/` through `Group52_10/`), 271 commits and 240 recorded working sessions in `logs/agent_history.log`. That log is not a changelog: each entry records the commit before and after, whether the remotes were fetched, whether the work is reachable from `main()`, and how it was verified. Several sections of this report are drawn from it, including the failures below.

Period	Dates	What landed
W04	29 May – 4 Jul	Environment and architecture. Game loop and `GameStateManager`, collision primitives and spatial hash, math utilities, the entity skeletons. Member B started the input and sound spine.
W05	5 – 11 Jul	Physics made real: camera, momentum and friction centralised into `PhysicsEngine`, wall sliding, and the JSON `LevelLoader`. Member B's gameplay entities, blocks and the animation system.
W06	12 – 18 Jul	Persistence and tooling: save slots, statistics, achievements, the pause and settings panel, and the first version of the level editor with its undo/redo Command framework.
W07	19 – 25 Jul	First real integration of both members' branches, the consolidated `verify_all` runner, the entity factory pipeline, and the design for two-player and Shadow Mario.
W08	26 Jul – 1 Aug	Particles on an object pool; repository synchronisation after the branches had diverged.

Period	Dates	What landed
W09	2 – 8 Aug	Fireballs through the pool, flagpole scoring, trampolines, checkpoints, the Memento time-rewind, and the first procedural generator.
W10	9 – 15 Aug	The three-world campaign with warp-pipe sub-levels, physics normalisation for static bodies, and hitbox tuning.
Delivery	16 – 22 Aug	The audit and its remediation, the first green CI, the boss fights, multiplayer and the CPU opponent, and the Windows-playtest defects in §10.

12.1 The audit, and what it cost

On 18 August a full review of both domains produced **37 findings, 7 of them critical** (`docs/issues/code_audit_2026-08-18.md`, GitHub issue #11). It is the hinge of the project, and three things about it are worth more than the findings themselves.

> ***The audit's first revision was wrong, and had to be retracted publicly.*** *It was written against a local* `dev` *that was nine commits stale, and on that basis declared the entire audio system missing. The audio system was implemented and merged on* `origin/dev`*. The finding was withdrawn on issue #11.*
>
> ***About 3,100 lines of finished work were sitting uncommitted.*** *Three sub-level maps, the tools directory and a week's report were in the working tree with no commit, one careless* `git clean` *from being lost. They were committed before anything else happened.*
>
> ***Six subsystems were "complete" and inert.*** *The minimap, particle system, death effects and screen transitions all compiled, all had passing harnesses, and none were constructed by the game. They were wired in the same session — and the menu music, which had been silently failing at startup for weeks, was found by* running the game for six seconds*, not by any test.*

Each of those became a rule rather than a patch, recorded in `AGENTS.md` with the incident that motivates it: fetch before describing repository state; never discard uncommitted work to unblock a

git operation; and "complete" means reachable from `main()` and observed running, not that a file exists and compiles. The report you are reading is written to that third rule, which is why §4 says how each capability was confirmed and §13 says what is not done.

12.2 What the history actually shows

The defect record has a shape. Of the five problems in §10, three are the same failure wearing different clothes — **one fact stored in two places, drifting apart, with nothing comparing them**. The entity list in the factory, the parser and the editor palette. The tile name-to-enum mapping duplicated in the loader, which silently dropped every coin tile in all seven level files. The flagpole's collision height (300) against its sprite height (168). None of these produced a compiler error; all of them produced wrong behaviour that looked like a design choice.

The project's answer, now standing practice, is that **the commit which creates the second copy also creates the test that fails when the copies disagree**. The entity catalogue's parity test (§8.3) is the model. The counterexample is instructive too: this very report is generated by a script that counts what it claims, and doing so caught two wrong figures before they were printed — a class count inflated by a substring match, and "23 test files" reported as if it meant "23 things CI runs".

The second pattern is thinner: **86 test harnesses against 4 recorded playtests** at the time of the audit. Every defect in §10.1, §10.4 and §10.5 was invisible to the test suite and obvious within seconds of looking at the running game. The suite is not the problem — it is what makes the fixes stick — but it verifies what someone already thought to doubt.

13 Verification, CI and known gaps

The tree holds 23 verification harnesses. 22 are built by CMake and 13 of those are registered as CTest cases and run by GitHub Actions on every push to the integration branches — the difference is the harnesses that open a window and cannot run on a headless runner, which are compiled but not executed there. The registered suite is currently 13/13 green, with 497 individual checks in the regression binary alone. Cases are added whenever a defect escapes to the audit stage, so the suite is a record of every bug the project has shipped.

13.1 What is not done

- **No 3D renderer.** The 5-point rubric line is forfeited, deliberately (§4).
- **The Windows crash fix is not confirmed on Windows.** Both verification runs were on macOS, where the bug was already invisible. They show the fix breaks nothing; the argument that it cures the crash is the analysis in §10.1. A Windows playtest of 1-3 is the outstanding confirmation.
- **The rebalanced Bowser fight is unplaytested.** The stagger cost (four fireballs) and its duration (three seconds) are reasoned, not measured against a player.
- **The test suite is not hermetic.** It reads and writes the real `saves/` directory; one run was observed deleting campaign progress, and one high-score assertion passes or fails depending on what is already on disk. This violates the project's own CI rule and is open work.
- **Playtest coverage is thin.** 4 recorded playtests against 86 test harnesses is the project's standing imbalance, and the reason several of the defects in §10 survived as long as they did.

14 Conclusion and future work

The project delivers a complete, playable platformer whose value as coursework lies in its architecture: ten design patterns, each introduced because a concrete problem demanded it, over a four-level abstract hierarchy that the engine drives without ever asking what an object is. Adding a new enemy means one class and one catalogue row; adding a new level means a JSON file.

The more useful outcome is what the defects taught. Three of the five problems in §10 were the same failure in different clothes — a fact duplicated in two places, drifting apart, with nothing comparing them. The project's answer, now a standing rule, is that the commit which creates the second copy also creates the test that fails when the copies disagree. The fourth, the Windows crash, is a reminder that "it works on my machine" is exactly the evidence undefined behaviour is best at manufacturing.

14.1 Future work

- Make the test suite hermetic, then add an AddressSanitizer job to CI — the tool that would have found §10.1 on its first run.
- Wrap the entity list in a type whose `push_back` only the flush step can reach, so the §10.1 invariant is impossible to break rather than merely easy to keep.
- Playtest and tune the Bowser fight, and record enough playtests to correct the 86:4 imbalance.
- Finish the procedural generator's integration — it produces winnable levels today but is not part of the campaign.

15 References

- Repository: github.com/ndmhuy/SuperMarioGame
- `SPEC.md` — frozen behavioural specification (constants, schemas, entity rules)
- `AGENTS.md` — engineering rules, and the incident history behind each
- `docs/learning/mid-frame-entity-spawn-crash.html` — full analysis of §10.1
- SFML 3.0.2 — sfml-dev.org; Dear ImGui + ImGui-SFML; nlohmann/json
- Gamma, Helm, Johnson, Vlissides, *Design Patterns* (1994) — Factory, Singleton, Observer, State, Strategy, Command, Decorator, Memento, Template Method
- Nystrom, *Game Programming Patterns* — Object Pool, Game Loop, Component
- Fiedler, "Fix Your Timestep!" — the fixed-step accumulator in §8.1
- Assets are from *Super Mario Bros.* (Nintendo, 1985), used for non-commercial educational purposes only. All rights remain with their owners.

16 Appendix

16.1 Build and run

```
cd SuperMarioGame
mkdir -p build && cd build
cmake ..                  # fetches SFML, ImGui-SFML and nlohmann/json
cmake --build . -j8
ctest --output-on-failure # 13 verification targets
./SuperMarioGame
```

16.2 Controls

Action	**Player 1**	**Player 2**
Move	A / D or ← / →	← / →
Jump	W, ↑ or Space	↑
Crouch / ground pound	S or ↓	↓
Run (hold)	Left Shift	Right Shift
Fireball	F or J	M
Rewind time (hold)	R	—
Level editor	F1	—
Dev panel / debug console	F12 / ~	—
Pause	Escape	Escape

16.3 Figures at a glance

Measure	**Value**
C++ source files / headers	130 / 139
Lines of C++ (excluding third-party)	27,926
Concrete entity classes	5 players, 13 enemies, 13 items, 10 blocks
Design patterns	10

Measure	Value
Verification harnesses	23 source files, 22 built, 13 registered as CTest cases
Commits on the integration branch	271
Sound effects / music tracks	29 / 12
Achievements	12

16.4 UML diagrams, collected

Every class diagram in this report, in one place, at full size — not a redrawn summary, the same figures §6 discusses in context, collected here for a reader who wants the whole structure in one pass rather than hunting back through the prose. Each is regenerated from the current headers at build time by the same `gen_class_diagram.py` pass that draws the inline copies (§6's opening note), so this appendix cannot go stale relative to them the way a hand-drawn "master diagram" done once and forgotten would.

Figure	Root class	Discussed in	What it shows
6	`Entity`	§6.1	The four branches under the abstract root: Character, Item, Block, Projectile.
6a	`Item`	§6.2	Twelve concrete power-ups and pickups behind one `activate()` call.
6b	`Block`	§6.2	Ten concrete blocks, three of them (FallingPlatform, Thwomp) State machines in their own right.
7	`Character`	§6.3	Player and Enemy split, including the ShadowMario replay rival.

Figure	Root class	Discussed in	What it shows
8	`Enemy`	§6.3	The enemy branch, with the Boss sub-hierarchy sealed by Template Method.
9	`IPlayerState`	§6.4	Five player forms plus the two Decorators that wrap them.
10	`IGameState`	§6.5	Eight screens behind one State interface.
11	`ICommand`	§6.5	Input as objects — the Command pattern's own hierarchy.
12	`IMovementStrategy`	§6.5	Eight interchangeable enemy movements — the Strategy pattern's own hierarchy.

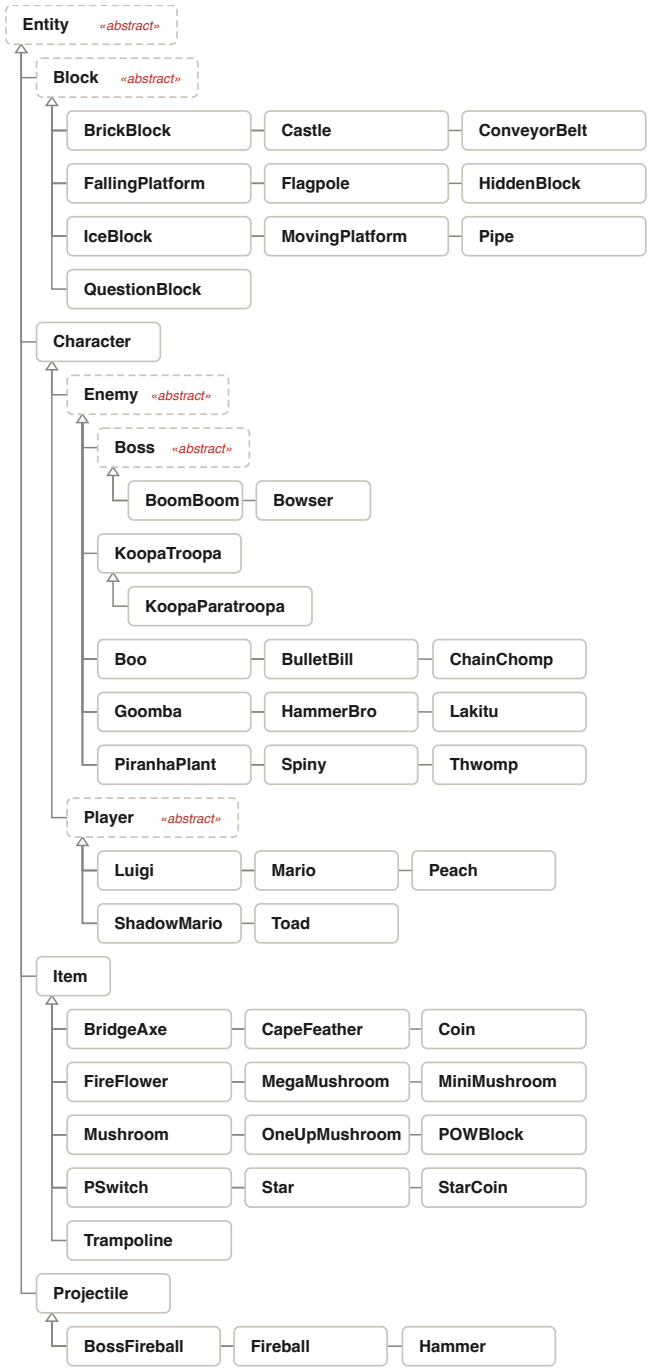

Figure 6 (full) — The complete Entity hierarchy.

Every concrete Entity subclass the game has, four branches deep from one abstract root.

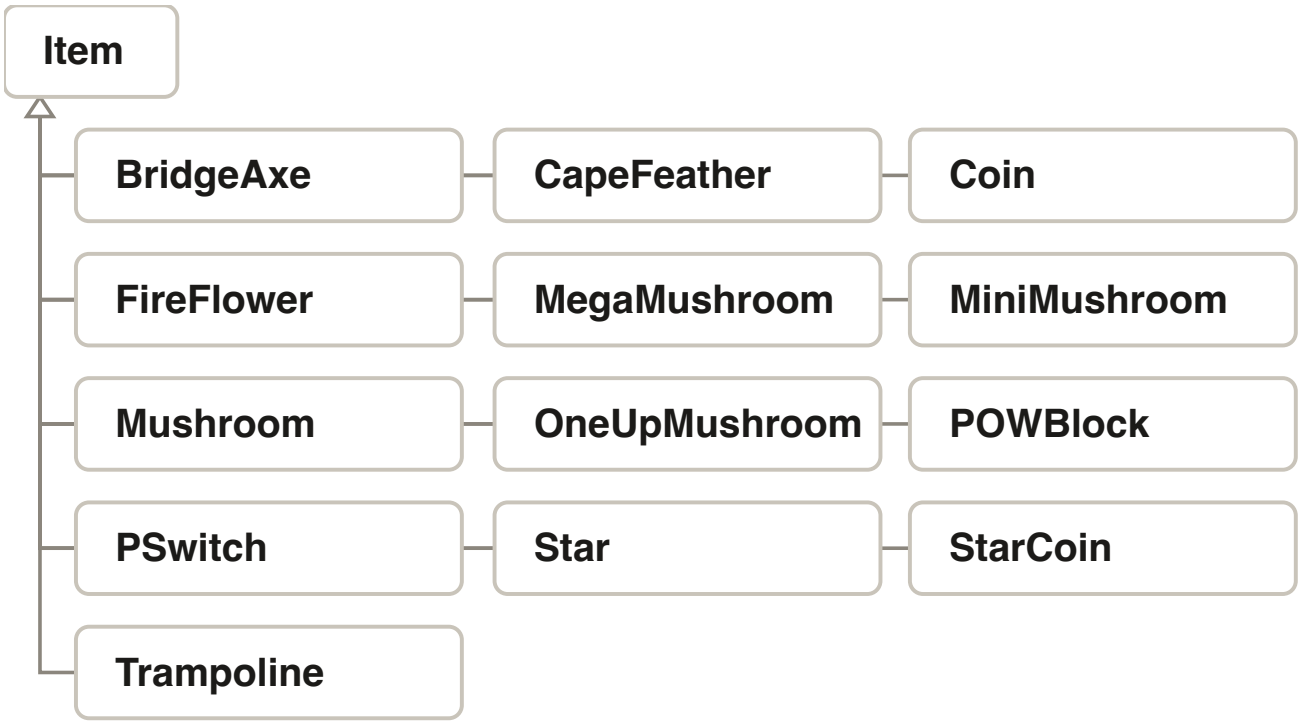

Figure 6a (full) — The complete Item hierarchy.

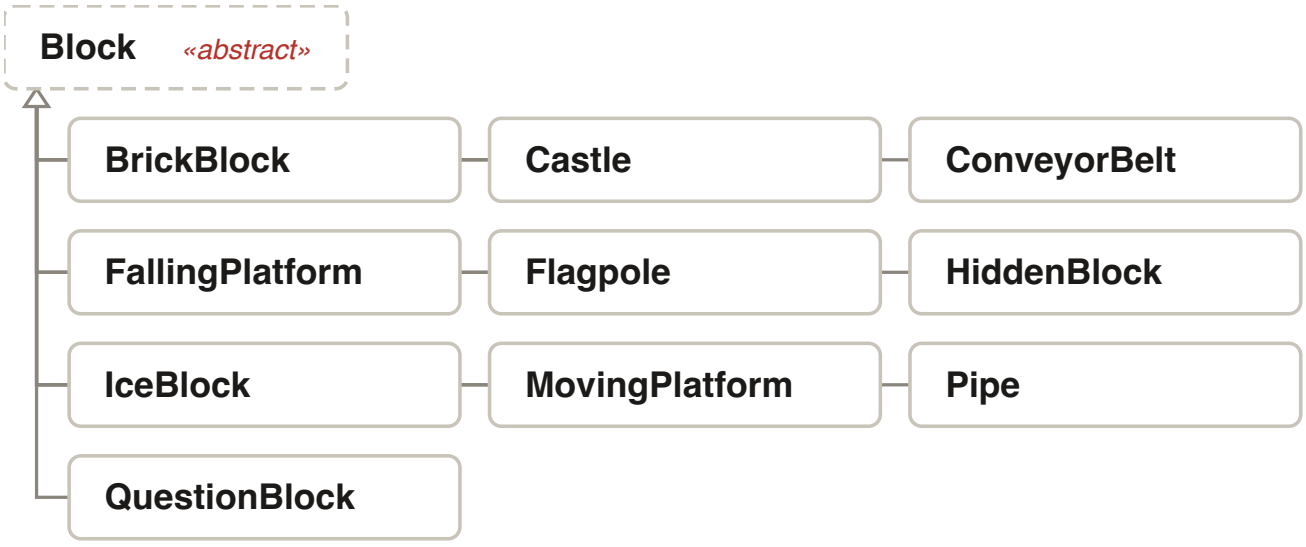

Figure 6b (full) — The complete Block hierarchy.

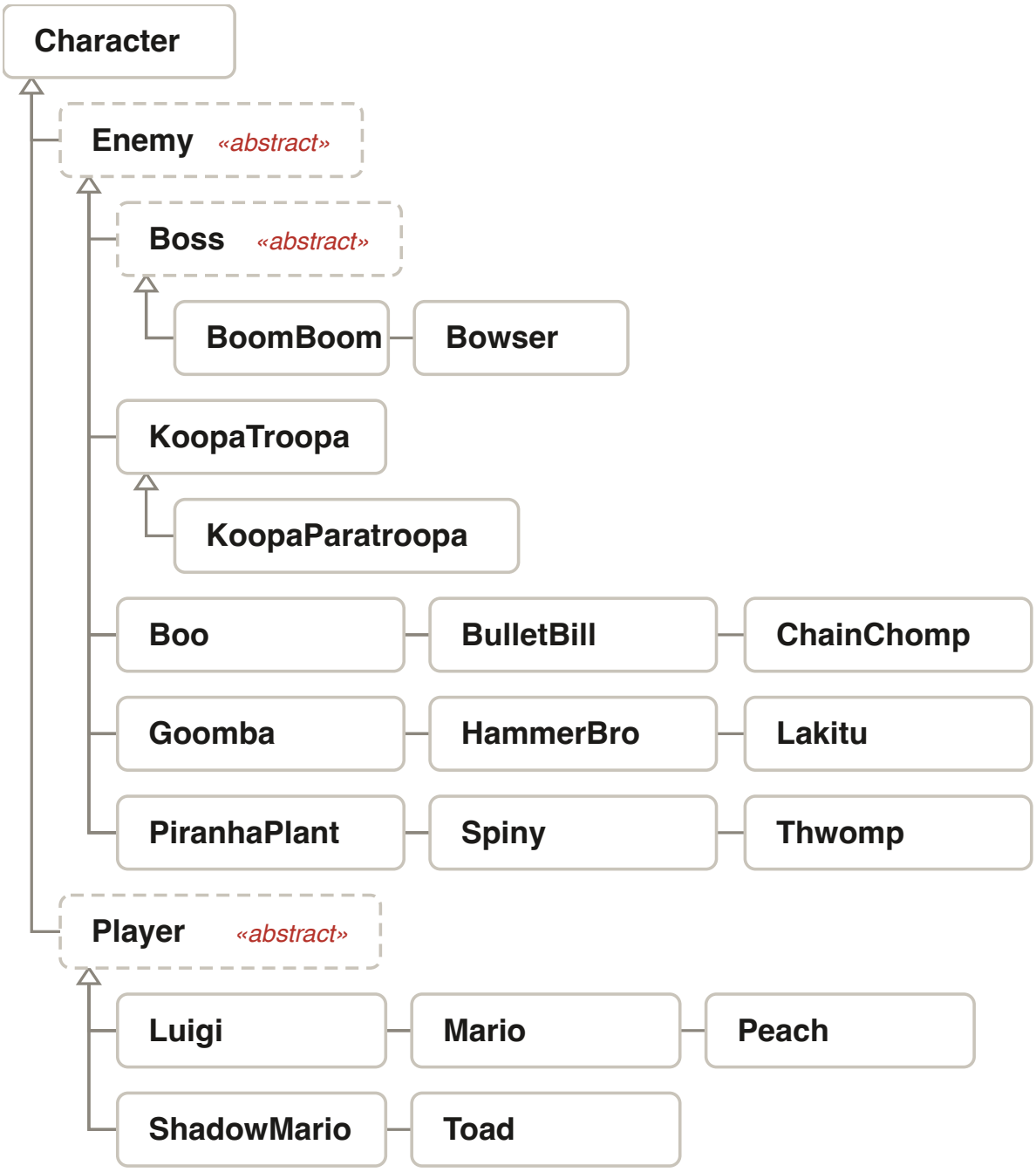

Figure 7 (full) — The complete Character hierarchy.

Every playable character and every enemy, including the Boss sub-branch, in one tree.

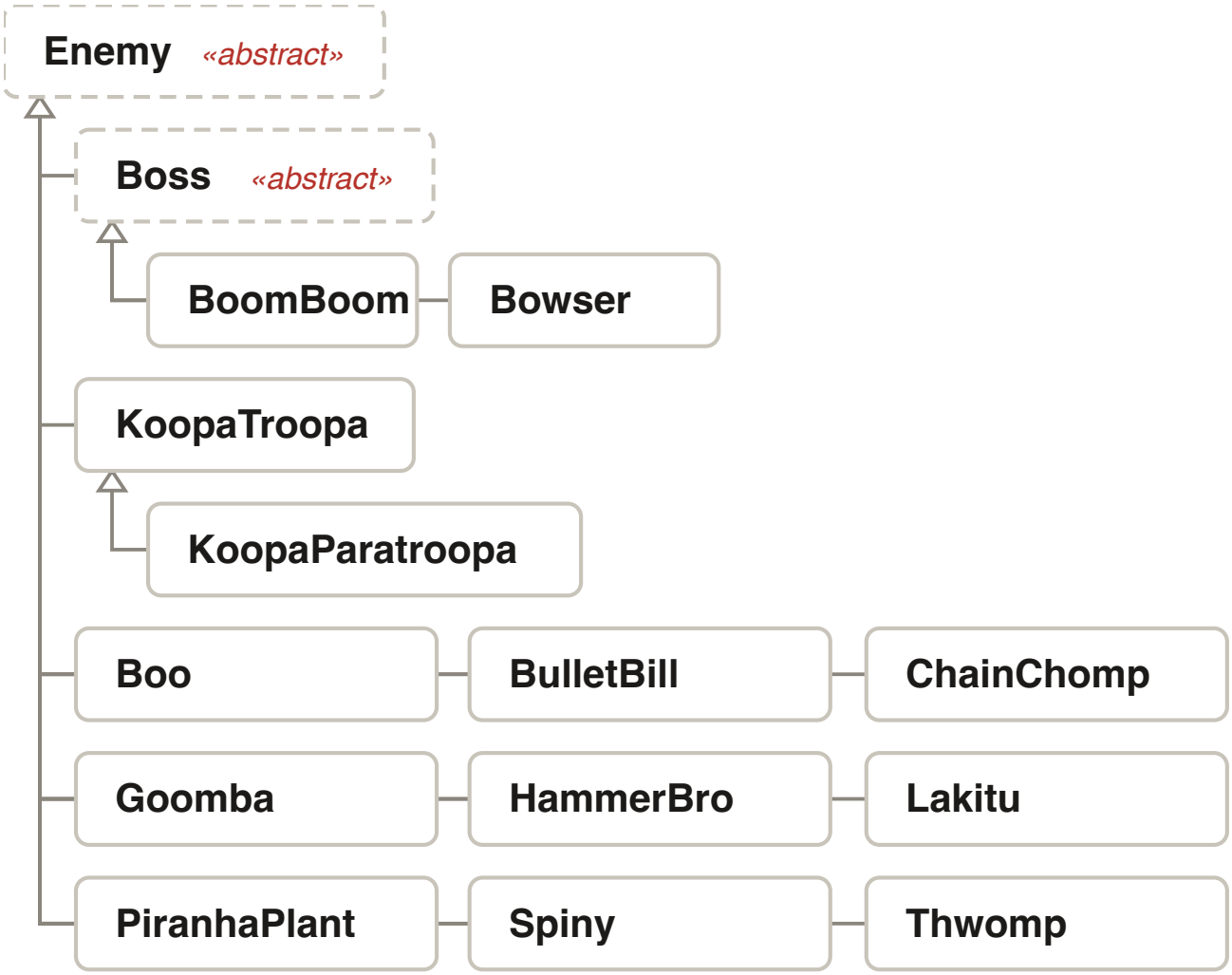

Figure 8 (full) — The complete Enemy/Boss hierarchy.

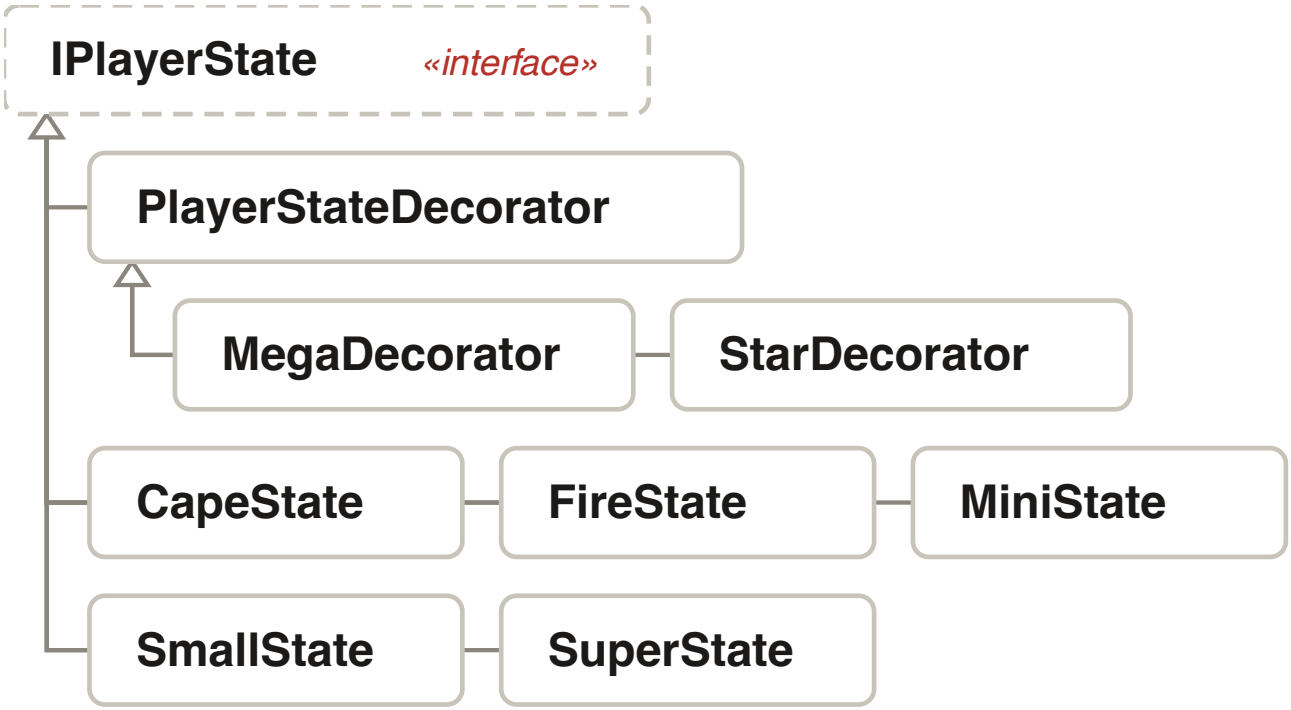

Figure 9 (full) — The complete player-form hierarchy.

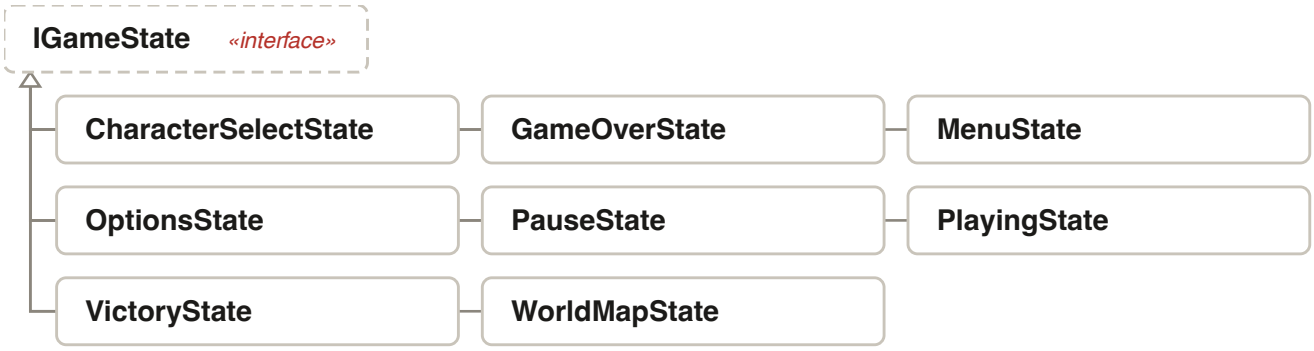

Figure 10 (full) — The complete screen/State hierarchy.

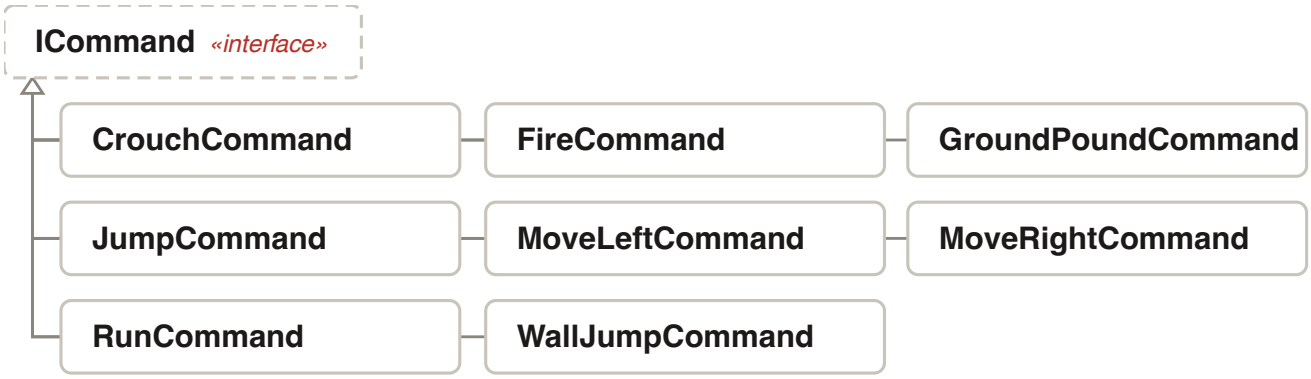

Figure 11 (full) — The complete Command hierarchy.

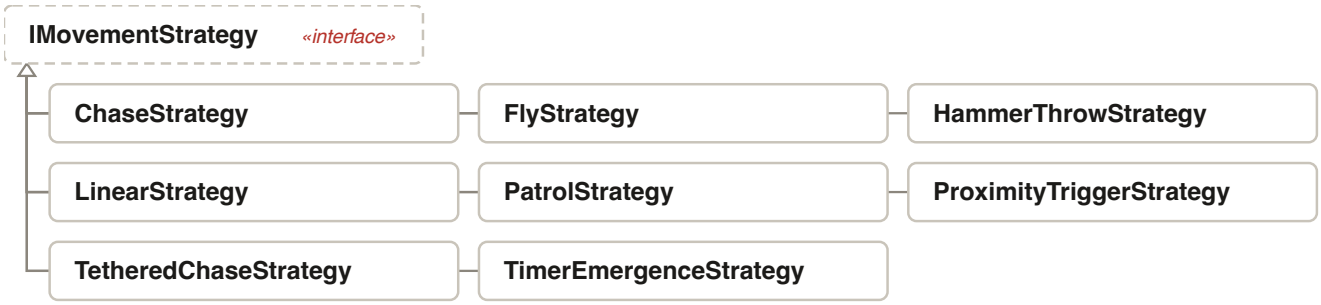

Figure 12 (full) — The complete Strategy hierarchy.

Group 52 · CS202 Object-Oriented Programming · University of Science, VNU-HCM

Generated by `reports/build_report.py` from the repository at commit 38bab75. Self-contained: opens from disk, no network, prints to PDF.